

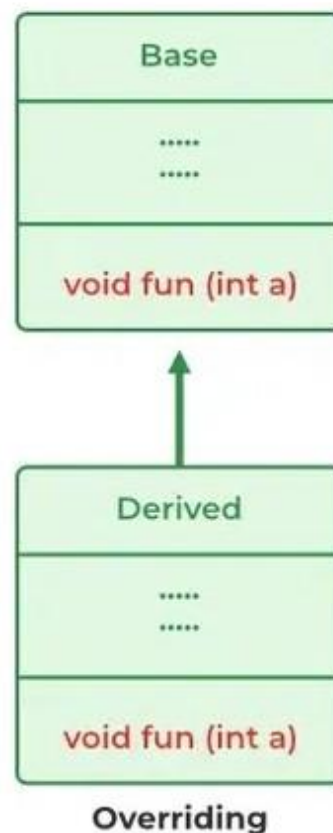
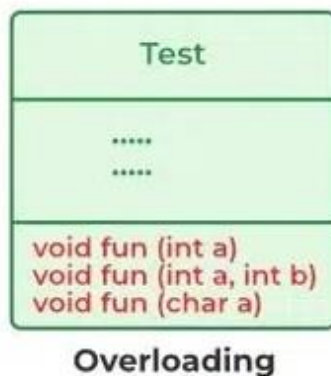
OOPS Concept

2. Method Overriding in Java :-

- Method Overriding in Java occurs when a subclass provides its own implementation of a method defined in the parent class.
- The method in the child class must have the same name, parameters, and return type as in the parent class.
- It is used to achieve runtime polymorphism.
- Only inherited methods can be overridden.

Rules for Method Overriding :-

- Same method name, parameters, and return type (or covariant).
- Must be in a subclass.
- Access modifier: cannot be more restrictive, can be wider.
- Static, final, private, and constructors → cannot be overridden.
- Exceptions: child can throw unchecked; for checked, same or subclass only.
- Supports runtime polymorphism (method call depends on object type).
- Use @Override for correctness.



Example –

```
package oops;

class Amitabh_bachhan {
    void info()
    {
        System.out.println("Age:82");
        System.out.println("Wife Name:Jaya");
    }
}

public class Abhishek_bachhan extends Amitabh_bachhan{
    void info()
    {
        super.info();
        System.out.println("Age:50");
        System.out.println("Wife Name:Aishwarya");
    }

    public static void main(String[] args) {
        Abhishek_bachhan ob=new Abhishek_bachhan();
        ob.info();
    }
}
```

Output –

```
Age:82
Wife Name:Jaya
Age:50
Wife Name:Aishwarya
```

Task –

```
package practical;

class company{
    void details() {
        System.out.println("Company Name:Capgemini");
    }
}

public class employee extends company {
    void details() {
        super.details();
        System.out.println("Name:Sagar Kate");
        System.out.println("Salary:5.5LPA");
    }

    public static void main(String[] args) {
        employee e = new employee();
        e.details();
    }
}
```

Output –

```
Company Name:Capgemini
Name:Sagar Kate
Salary:5.5LPA
```

Difference Between Method Overriding & Method Overloading :-

Features	Method Overriding	Method Overloading
Definition	Method overriding is about the same signature in subclass.	Method overloading is about same method name, different parameters.
Polymorphism	It is also known as Runtime polymorphism	It is also known as Compiletime polymorphism
Inheritance	Requires inheritance.	Can be in the same class or subclass
Return Type	Return type must be same	Return type can be different
Exceptions	Must follow overriding rules.	No restrictions.

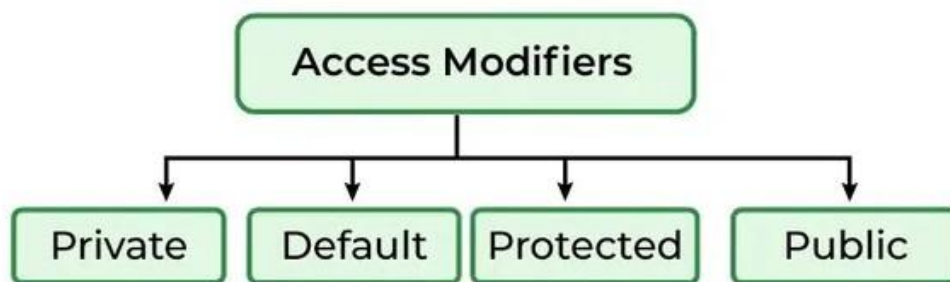
Access Modifiers in Java :-

- In Java, access modifiers control the visibility of class members.
- They apply to variables, methods, and classes.
- Access modifiers determine how and where these members can be accessed.
- They help ensure security and encapsulation.
- They support modular and maintainable code in large applications.

Types of Access Modifiers :-

There are 4 types of access modifiers available in Java:

Access Modifiers in Java



- 1. Default** : No keyword required and accessible within the same package.

Example –

```
class DefaultExample {  
  
    int num = 20; // default variable  
  
    void display() { // default method  
        System.out.println("Default: " + num);  
    }  
  
    public static void main(String[] args) {  
        DefaultExample obj = new DefaultExample();  
        obj.display(); // Works  
    }  
}
```

2. Private : Only accessible within the class.

Example –

```
class PrivateExample {  
    private int num = 10; // private variable  
  
    private void display() { // private method  
        System.out.println("Private: " + num);  
    }  
  
    public static void main(String[] args) {  
        PrivateExample obj = new PrivateExample();  
        obj.display(); // Works  
        // System.out.println(obj.num); // Error: num is private  
    }  
}
```

3. Protected : Accessible within the class and its subclasses.

Example –

```
class ProtectedExample {  
    protected int num = 30; // protected variable  
  
    protected void display() { // protected method  
        System.out.println("Protected: " + num);  
    }  
}
```

```
class SubClass extends ProtectedExample {  
    public static void main(String[] args) {  
        SubClass obj = new SubClass();  
    }  
}
```

```
    obj.display(); // Works: inherited protected method
    System.out.println(obj.num); // Works: inherited protected variable
}
}
```

4. Public : Accessible from anywhere

Example –

```
public class PublicExample {
    public int num = 40; // public variable

    public void display() { // public method
        System.out.println("Public: " + num);
    }

    public static void main(String[] args) {
        PublicExample obj = new PublicExample();
        obj.display(); // Works
        System.out.println(obj.num); // Works
    }
}
```

final Keyword in Java :-

- The final keyword in Java is a non-access modifier.
- It is used to prevent modification.
- The following are different contexts where the final is used:

Final Variable  **To Create constant variable**

Final Methods  **Prevent Method Overriding**

Final Classes  **Prevent Inheritance**

1. Variable -

- A final variable's value cannot be changed once it is initialized.
- It acts as a constant in the program.
- Final variables must be initialized when declared or in the constructor.

Example –

```
class FinalVariableExample {  
  
    final int num = 10; // final variable  
  
  
    public static void main(String[] args) {  
  
        FinalVariableExample obj = new FinalVariableExample();  
  
        System.out.println(obj.num);  
  
        // obj.num = 20; // Error: cannot change final variable  
    }  
}
```

2. Method –

- A final method cannot be overridden by subclasses.
- It ensures the behavior of the method remains unchanged.
- Useful for methods that are critical and should not be modified.

Example –

```
class Parent {  
  
    final void display() {
```

```
        System.out.println("Final method in Parent");
    }
}
```

```
class Child extends Parent {
    // void display() {} // Error: cannot override final method
}
```

3. Class –

- A final class cannot be inherited.
- It prevents subclassing, ensuring the class's implementation is fixed.
- Useful for security and immutability, like the String class.

Example –

```
final class FinalClass {
    void show() {
        System.out.println("This is a final class");
    }
}
```

```
// class SubClass extends FinalClass {} // Error: cannot inherit final class
```